

HybridSYN: An Efficient Logic Synthesis Methodology

1st Ke Wang

Hangzhou Dianzi University
School of Electronics Information
Hangzhou, China
231040081@hdu.edu.cn

2nd Yue Wu*

Hangzhou Dianzi University
School of Electronics Information
Hangzhou, China
yuewu@hdu.edu.cn

3rd Xiaoyan Yang

Hangzhou Dianzi University
School of Electronics Information
Hangzhou, China
yangxiaoyan@hdu.edu.cn

Abstract—As the complexity of modern circuits increases, logic synthesis faces growing challenges. Reinforcement learning (RL) shows promise for efficient logic optimization. However, current RL methods rely solely on a single Boolean circuit representation—typically And-Inverter Graphs (AIG). This constraint hinders the exploration of larger optimization spaces, leading to suboptimal outcomes. This study introduces HybridSYN, a hybrid RL framework combining optimization algorithms from AIG and Majority-Inverter Graphs (MIG) to address the limitations of synthesis methods relying on a single logic graph representation. By expanding the action space, HybridSYN enables the exploration of better LUT-level solutions. To improve state representation, HybridSYN processes topological features from both AIG and MIG using independent Graph Convolutional Network (GCN) encoders. The resulting features are fused and passed through a fully connected network, where the final layer jointly predicts the type of operator and its parameters. Experimental results on the EPFL benchmark suite show that HybridSYN achieves a 9.7% reduction in LUT count compared to existing RL methods and a 17.1% improvement over baseline approaches.

Index Terms—Reinforcement Learning, Logic Synthesis, Logic Optimization, Electronic Design Automation.

I. INTRODUCTION

Logic synthesis is a crucial step in circuit design, as it significantly improves design efficiency and performance by converting the abstract behavior of a circuit into an optimized gate-level implementation. Its primary functionality lies in automatically optimizing the structure and delay of a circuit, thereby enhancing the overall system's performance. Traditionally, solutions to logic synthesis problems were based on heuristic algorithms [1] and manual processes. As circuit complexity increases, traditional heuristic-based approaches rely on predefined optimization sequences. These sequences lack adaptability to diverse circuit characteristics and face exponentially growing search spaces, making them prone to local optima and insufficient for comprehensive design space exploration [2], [3].

To address these limitations, RL-based methods have been increasingly utilized in logic synthesis. Early RL approaches focused on leveraging the And-Inverter graph (AIG) for circuit optimization, which made progress in the synthesis sequence exploration [4]–[6]. With the emergence of Majority-Inverter graphs (MIG) as an alternative representation for arithmetic-

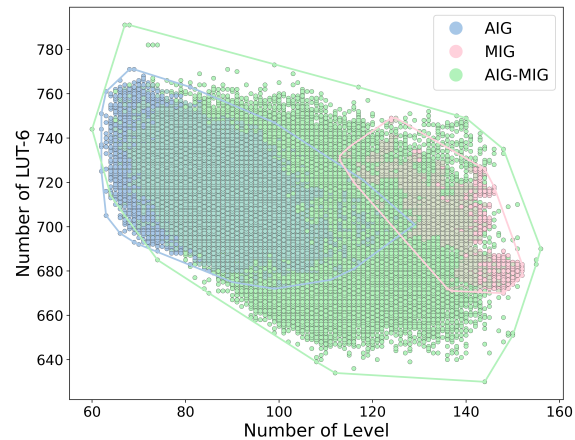


Fig. 1. Distribution of Level-LUT for 50,000 randomly generated synthesis flows on the max circuit: AIG, MIG, and AIG-MIG hybrid.

intensive designs, recent advancements in RL have shifted towards leveraging the operators of MIG to optimize circuits [7], [8].

As shown in Fig. 1, the design results for "max" are distributed differently. We used 50,000 randomly generated synthetic scripts on the EPFL benchmark suite [9] with ABC [10] and Mockturtle [11], [12] tools. The exploration space varies significantly, and this depends on the logic representation. This shows that hybrid exploration of both AIG and MIG can facilitate a more thorough search and ultimately lead to better Quality of Results (QoR). However, the search space available for exploration is inherently different for each logic representation, and relying on one representation alone often restricts the potential for discovering optimal solutions.

To overcome these problems, we propose an RL framework for automatic logic optimization using the Proximal Policy Optimization (PPO) algorithm. Named HybridSYN (Hybrid AIG-MIG Logic Synthesis), this framework explores the search space and generates optimized operator sequences with PPO. The main contributions are summarized as follows:

- We propose a hybrid optimization model based on PPO, which leverages both AIG and MIG representations along with their corresponding optimization algorithms.

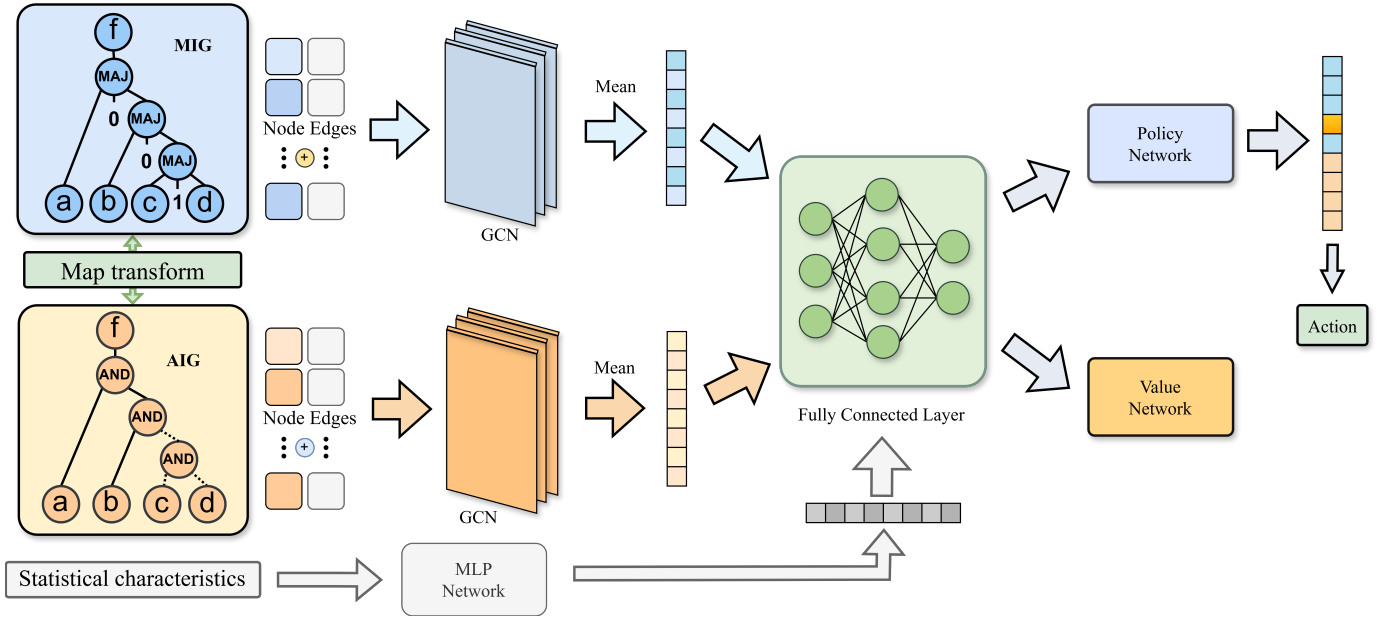


Fig. 2. Network Architecture of our method

- We develop a unified optimization environment with seamless AIG/MIG conversion capabilities, enabling RL-driven selection of optimal representations and operator sequences.
- A cross-attention mechanism fuses features from AIG and MIG, along with circuit scalar metrics, to create a more comprehensive state representation.
- On the EPFL arithmetic benchmark suite [9], compared to previous techniques [6], the proposed method can reduce the number of look-up tables (LUTs) by 9.7% without increasing the average levels.

II. FUNDAMENTALS AND RELATED WORK

A. Reinforcement Learning Algorithms

We adopt PPO2 [13], a standard method for policy optimization in reinforcement learning. PPO2 optimizes a surrogate loss defined as:

$$L(\theta) = \mathbb{E} \left[\min (r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (1)$$

where $r_t(\theta)$ is the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$, and $\hat{A}_t$ is the advantage function at time t . The advantage function $\hat{A}_t$ is used to estimate the relative advantage of taking action a_t over the current policy. To calculate this advantage, we use the Generalized Advantage Estimation (GAE) [14], which is defined as:

$$\hat{A}_t = \sum_{l=0}^{\infty} \gamma^l (r_{t+l+1} - V_t) \quad (2)$$

where γ is the discount factor and V_t is the value function estimated by the critic network at time step t .

B. Logic Representation

In digital circuit design, Boolean networks are commonly represented as a directed acyclic graph (DAG). In this context, each node in the DAG represents a logical operation or a primary input/output (PI/PO), and each directed edge represents a connection between them. Circuit representations such as AIG and MIG are widely used to model and optimize these Boolean networks.

AIG is a widely used Boolean network representation, consisting of two input AND gates and inverters organized in a DAG. It provides an efficient structure for representing Boolean functions and facilitates the optimization of logic circuits. AIG has become a standard representation in many logic synthesis tools due to its compactness and ease of manipulation.

In recent years, MIG has become a promising alternative to AIG, especially for optimizing arithmetic-intensive designs. This is because MIG uses majority nodes and inverters to implement the majority function, which generalizes the AND and OR operations. The expressive majority function enables powerful optimization techniques, such as logic simplification, delay balancing, and cost reduction. Consequently, these characteristics make MIG exceptionally effective in arithmetic circuits, leading to improved QoR [15], [16].

C. Cross Attention for Feature Fusion

Cross-attention is a powerful mechanism used for feature fusion, especially when combining different feature sets. It enables the model to learn the relationship between distinct features by assigning weights to them based on their relevance.

Let Q be the query, K be the key, and V be the value. The attention score is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

where d_k is the dimension of the key vector. In our framework, we use features from one graph as queries and from the other as key-value pairs, allowing the adaptive fusion of AIG and MIG embeddings.

D. Related Work

Previous studies cast logic synthesis as a machine-learning sequence optimization task. DRILLS [4] introduces an advantage actor-critic agent to minimize logic network area under delay constraints. This approach is applied to optimize AIG representations. Similarly, RL-A2C [5] generates synthesis scripts matching the *resyn2* baseline in length, focusing on AIG optimization. Zhou et al. propose a PPO-based logic synthesis framework [6] for AIG with an analysis of statistical features, which further contributes to AIG optimization. Peruvemba et al. present a runtime-constrained RL method [7] using a logic network reward function and termination conditions to generate synthesis scripts, validated in separate AIG and MIG environments. The ESE framework [8] proposed by Yu Qian et al. uses RL to generate customized synthesis scripts for circuits, with experiments conducted separately in AIG and MIG.

However, existing RL-based methods generally optimize a single logic network representation. AIG and MIG exhibit complementary strengths: AIG achieves superior area reduction in control-intensive circuits, while MIG significantly reduces logic depth in arithmetic-dominated circuits, as supported by prior works [8], [17]. To leverage the unique advantages of both representations, we propose a hybrid optimization approach that combines AIG and MIG primitives within a unified synthesis flow.

III. METHODOLOGY

A. Framework

The proposed framework, *HybridSYN*, alternates between AIG and MIG representations to optimize circuits. Training begins by initializing the encoder E_ϕ , critic V_ω , actor π_θ , and a replay buffer. The method is outlined in [Algorithm 1](#). At the start of each episode, the logic synthesis environment is reset. At every step t , *HybridSYN* encodes circuit features into an RL state s_t via the encoder E_ϕ . The actor, implemented as a PPO-based policy network, selects an optimization action a_t , which updates the circuit and transitions it to a new state s_{t+1} . The critic evaluates the action a_t by computing the reward r_t and estimating the value function v_t . The transition tuple $(s_t, a_t, r_t, v_t, s_{t+1})$ is stored in the replay buffer. This process is repeated until the synthesis sequence reaches a predefined length L , at which point the trajectory is stored. Once t_u trajectories are collected, the networks are updated, the replay buffer is cleared, and training continues until the maximum number of episodes, $max_episode$, is reached.

Algorithm 1 Learning Framework to Explore Logic Synthesis

Input: Boolean network

Output: Synthesis script

```

1: Initialize encoder  $E_\phi$ , critic  $V_\omega$ , and actor  $\pi_\theta$  with random
   parameters  $\phi, \omega, \theta$ .
2: Initialize  $max\_episodes$ ,  $L$ ,  $t_u$ , and replay buffer
    $ReplayBuffer$ .
3: for episode = 1 to  $max\_episode$  do
4:   Reset Logic Synthesis Environment (HybridSYN Env).
5:   for t = 1 to  $L$  do
6:     Observe state:  $s_t = E_\phi(\text{AIG}, \text{MIG}, \text{features})$ .
7:     Select action:  $a_t \leftarrow \pi_\theta(s_t)$ .
8:     Execute action  $a_t$ , observe next state  $s_{t+1}$  and reward
        $r_t$ .
9:     Estimate value:  $v_t \leftarrow V_\omega(s_t, s_{t+1}, r_t)$ .
10:    Store  $(s_t, a_t, r_t, v_t, s_{t+1})$  in  $ReplayBuffer$ .
11:  end for
12:  if  $(episode + 1) \bmod t_u = 0$  then
13:    Update networks using data from  $ReplayBuffer$ .
14:    Clear  $ReplayBuffer$ .
15:  end if
16: end for

```

B. Environment Setup

To facilitate dynamic representation switching between AIG and MIG, we develop an RL environment for logic synthesis, leveraging the widely-used open-source tools ABC [10] and Mockturtle [11], [12]. Our environment leverages efficient transformation methods [18] to ensure seamless conversion between AIG and MIG while preserving the quality of results. Additionally, it supports the concurrent selection of AIG and MIG logic graph representations and their corresponding optimization algorithms, enabling flexible and efficient optimization by switching between the two.

1) State: The state of the environment is divided into three parts: two parts describe the graph features of AIG and MIG, while the other part consists of a feature vector that includes the following values at step t:

- Number of nodes, logic depth, and edges of the circuit.
- 6-LUT count and level after the circuit gets mapped.
- The current of the operator sequence.

2) Action Space: To demonstrate the feasibility of our approach, we selected the same 7 actions as those in ABC's *resyn2*, $A_{aig} = \{\text{balance}, \text{rewrite}, \text{rewrite -z}, \text{refactor}, \text{refactor -z}, \text{resub}, \text{resub -z}\}$, as well as the 6 actions in Mockturtle for MIG, $A_{mig} = \{\text{rewrite}, \text{rewrite -z}, \text{refactor}, \text{resub}, \text{resub -z}, \text{balance}\}$. Our model utilizes the union of these two action spaces, i.e., $A_{total} = A_{aig} \cup A_{mig}$.

3) Reward Function: To enable the agent to learn designs with varying LUT and level counts, a reward function based solely on a linear relationship with these quantities is inadequate. Thus, a normalized reward function is employed. Additionally, the normalization of the level count is incorporated

TABLE I
COMPARISON OF 6-LUT COUNTS AND LUT DEPTHS ON THE EPFL ARITHMETIC BENCHMARK SUITE, WITH THE IMPROVEMENT PERCENTAGE REPRESENTING THE OPTIMIZATION RATIO OF 6-LUT COUNTS RELATIVE TO *resyn2*.

Circuit	<i>resyn2</i>	DRiLLS [4]	RL-A2C [5]	RL-PPO [6]	Ours
max	719(78)	694	693	687.8(112.0)	661.2 (112.1)
adder	249(121)	244	244	244(116)	192.5 (64)
cavlc	118(5)	112.2	111.7	111.3(6.1)	110.3 (6.5)
ctrl	29(2)	28	28.2	28(2)	26.7 (2.1)
int2float	46(5)	42.6	42.7	42.3(5.0)	41.7 (5.7)
router	76(8)	70.1	69.9	69.5(8.4)	45.3 (7.5)
priority	220(86)	133.4	145.9	142.9(24.8)	149.3(29)
i2c	320(8)	292.1	290.5	289.3(8.1)	286.2 (6.2)
sin	1466(76)	1441.5	1438.2	1438(74.5)	1435.1 (72.2)
square	3915(122)	3889.4	3888.2	3889(121.7)	3767.4 (121.6)
sqrt	5127(2216)	4708	4698.8	4685.3(1804.6)	3847.4 (1844.4)
log2	7703(153)	7583.6	7580.3	7580.1(159)	7461.4 (153.3)
multiplier	5712(2216)	5678	5672.3	5672(126)	4982.3 (127.8)
voter	1828(20)	1834.7	1686.2	1678.1(19.06)	1657.4 (20.3)
div	8197(2060)	7944.4	7639.8	7807.1(2063.3)	4230.1 (2084.7)
mem	11459(51)	10527.6	10363.5	10309.7(45.6)	10223.9 (52.4)
GEOMEAN	808.78(51.1)	753.5	750.7	748.6(48.0)	670.8 (46.8)
ratio	1(1)	0.932	0.928	0.926(0.939)	0.829 (0.916)

to encourage the agent to optimize for solutions that meet the delay constraints. The reward function is defined as follows:

$$r_t = \frac{Lut_{init} - Lut_t}{Lut_{init}} + \frac{Lev_{init} - Lev_t}{Lev_{init}} \quad (4)$$

C. Network Architecture

We introduce a Graph Convolutional Network (GCN) architecture to enhance RL-driven optimization in logic synthesis. As shown in Fig. 2, the pipeline begins by preprocessing the features of each circuit node. These include the inverter count (#inverter) and a learned node-type embedding. A fully connected (FC) layer then projects them into a shared latent space.

For feature extraction, we apply two successive GCN layers to each graph type. Each GCN layer is followed by batch normalization (BN) and a ReLU activation to ensure stable training and nonlinear transformation. To capture structural patterns at the graph level, we use dual pooling operations: global mean pooling for overall feature distributions and global max pooling to highlight salient local patterns. The pooled vectors from both graphs are then fused using a cross-attention mechanism, which models inter-graph dependencies. The resulting fused vector is passed through a fully connected (FC) layer to align its dimensions. We concatenate this fused graph embedding with numerical state features and feed them into shared neural layers. The unified representation drives two heads: a Policy Network, which outputs a softmax distribution over discrete optimization operators, and a Value Network, which linearly projects to a scalar state value for PPO’s advantage calculation. By coordinating discrete operator selection with continuous parameter tuning, this framework enables efficient logic synthesis optimization.

IV. EXPERIMENTAL RESULTS

A. Benchmarks and Environments

We implemented the HybridSYN environment using ABC [10] and Mockturtle [11], [12], along with a C++ interface. Our method was developed in Python v3.8.8 with PyTorch v1.13.0. The experiments were conducted on a machine equipped with two 28-core Intel Xeon Gold 6348 CPUs and a single NVIDIA GeForce RTX 4090 with 24GB of CUDA memory. The synthesis sequence length L was set to 25, and the number of iterations was 200. FPGA mapping was performed using “if -a -K 6” for AIG and “lutmap -a -K 6” for MIG, with the number of LUTs collected as the final metric. Experiments were conducted on EPFL benchmarks, and the results were compared with those reported in DRiLLS [4], RL-A2C [5], and RL-PPO [6], with all comparative data sourced from RL-PPO.

B. Main Results

Table I presents the comparison results for 6-LUT counts, with values in parentheses indicating the corresponding netlist levels. Bold numbers denote the best performance among all methods. The final column shows our model’s performance averaged over the last 10 training rounds, demonstrating stable convergence. We adopt the same evaluation protocol as RL-PPO to ensure a fair comparison. The primary objective is to minimize 6-LUTs while maintaining the number of levels satisfying the constraints.

Among the 16 benchmark circuits, our method achieved the best results in 15 cases. Compared to prior AIG-only optimization approaches, such as RL-A2C and RL-PPO, our method reduced 6-LUTs by 9.9% and 9.7%, respectively. The limited improvements in certain cases stem from the inherent constraints of single-representation optimization, which fails to adapt to diverse circuits, such as arithmetic-heavy or control-dominated designs. By incorporating both AIG and

TABLE II
EXPERIMENTAL RESULTS WITH THE AIG GRAPH OR THE MIG GRAPH.

Circuit	<i>resyn2</i>	w/o MIG	w/o AIG	w/o feature	our
max	719	689.3	694.3	677.8	661.2
adder	249	245.3	194.6	193.6	192.5
cavlc	118	112.1	117.5	116.3	110.3
ctrl	29	27.9	27.1	27	26.7
int2float	46	43.2	50.4	42.1	41.7
router	76	70.1	48.9	51.3	45.3
priority	220	148.6	199.4	153.3	149.3
i2c	320	290.4	320	292.4	286.2
sin	1466	1441.6	1444.7	1436.6	1435.1
square	3915	3891.5	3782.6	3777.6	3767.4
sqr	5127	4711.6	3879.4	3874.1	3847.4
log2	7703	7569.1	7482.3	7512.2	7482.3
multiplier	5712	5598.7	5024.3	5003.6	4976.4
voter	1828	1696.7	1784.1	1664.5	1657.4
div	8197	7819.2	8269.3	4305.4	4230.1
mem	11459	10328.4	10279.6	10237.1	10223.9
GEOMEAN	808.8	752.7	742.3	684.3	670.8
ratio	1	0.931	0.918	0.846	0.829

MIG optimizations, our approach overcomes these limitations, achieving further reductions in 6-LUTs. Moreover, our method yields an average depth of 46.8 levels—about 2.3% shallower than the depth achieved by the *resyn2* heuristic—while still reducing LUT count. These results underscore the advantages of hybrid circuit representations in RL-based logic optimization.

C. Ablation Study

We conducted two ablation studies to evaluate the impact of MIG optimization operators and MIG graph features. The results are summarized in Table II. In the first ablation study, the results labeled “w/o MIG” correspond to AIG optimization only. In this setting, we exclusively use AIG optimization operators A_{aig} . The “w/o AIG” setting excludes AIG optimization within the HybridSYN framework and uses only the MIG operator A_{mig} , its graph features, and statistical features. In the “w/o features” setting, we concatenate AIG and MIG features together without using cross-attention. In both cases, the rest of the HybridSYN framework remains unchanged, and PPO is used to select both AIG and MIG optimization algorithms. This setup allows us to compare the impact of excluding MIG features.

Experimental results show that optimization using only MIG operators outperforms AIG in 7 out of 16 benchmark circuits, particularly in arithmetic-intensive designs. This aligns with previous studies [17], which demonstrate that MIG’s majority node primitives inherently compress arithmetic logic more efficiently than AIG’s AND node structures. Conversely, AIG achieves superior 6-LUTs minimization in control-intensive circuits due to its effective local Boolean simplification. HybridSYN enables dynamic switching between AIG and MIG optimization algorithms based on circuit characteristics, allowing for more flexible optimization across diverse designs. Furthermore, the MIG feature extraction module improves the minimization of 6-LUTs by 1.7%, improving the overall efficiency of HybridSYN by refining circuit feature represen-

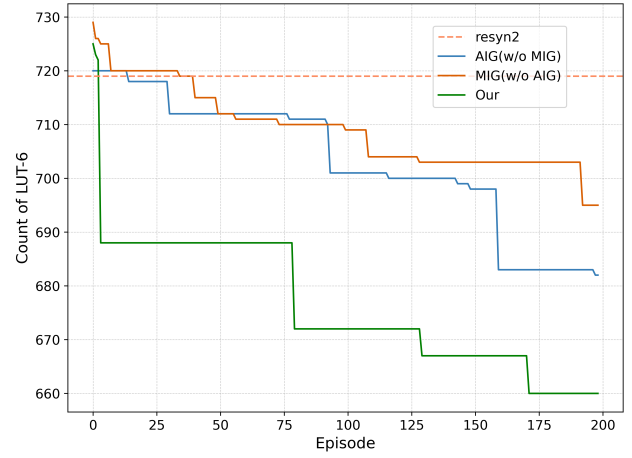


Fig. 3. Comparison of Iterative 6-LUT Minimization for AIG(DRILLS), MIG(w/o AIG), and Our(Hybrid AIG-MIG) Approaches over 200 Training Iterations in the Comprehensive Synthesis Flow for the Max Circuit

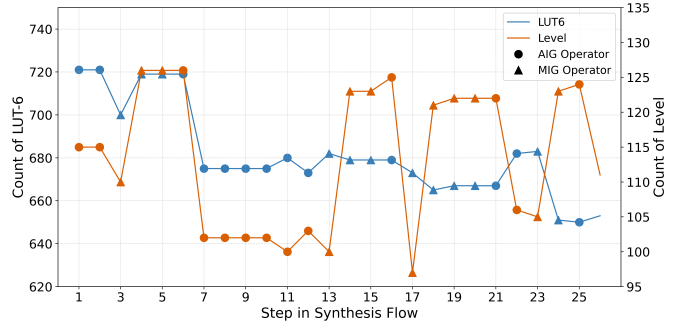


Fig. 4. Impact of Synthesis Flow on 6-LUT Count and Logic Level for the max Circuit: AIG and MIG Optimization Algorithms Effects over the Last 25 Steps

tation. These ablations confirm that performance gains stem from both switching between AIG and MIG representations and from expanding the operator action space.

D. Explore Efficiency Analysis

To evaluate the exploration efficiency of our model, we tracked the reduction in the 6-LUT count for the *max* circuit over 200 iterations of training. As shown in Fig. 3, the hybrid optimization approach combines AIG and MIG optimization algorithms, meeting logic depth constraints while achieving faster convergence to better solutions. Although better solutions than *resyn2* can be found within the exploration spaces of AIG or MIG, they do not do so as efficiently. This inefficiency arises from the limitations of each individual exploration space. The final synthesis steps shown in Fig. 4 highlight the synergy between AIG and MIG optimization algorithms. As optimization progresses, the reduction in the 6-LUT count stagnates after 17 steps. At this stage, switching to the other network representation enables further reduction in the 6-LUT count and logic depth. In summary, our approach enhances exploration efficiency by leveraging both AIG and

MIG optimization algorithms, allowing the model to escape local optima and achieve better optimization results.

V. CONCLUSION

HybridSYN introduces a novel RL framework that combines optimization algorithms from both AIG and MIG to address the limitations of existing RL-based methods that rely on a single logic graph representation. By expanding the action space and leveraging hybrid feature representations, HybridSYN facilitates a more comprehensive exploration of optimization strategies. Ablation experiments have confirmed the effectiveness of integrating AIG and MIG optimization algorithms. The integration of the action spaces of AIG and MIG facilitates a more efficient search for optimal solutions. Comprehensive evaluations on the EPFL benchmark suite confirm the framework's superior performance in logic synthesis sequence exploration, particularly in complex arithmetic circuits, where our method achieves 9.7% 6-LUT improvement over conventional approaches.

VI. FUTURE WORK

We will develop more efficient coordination strategies for AIG and MIG representations. These strategies aim to improve optimization effectiveness during multi-graph switching. We will also explore novel cross-modal fusion frameworks to enable better operator selection in logic synthesis.

REFERENCES

- [1] Xing Li, Lei Chen, Fan Yang, Mingxuan Yuan, Hong tao Yan, and Yupeng Wan. Himap: a heuristic and iterative logic synthesis approach. *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022.
- [2] Chang Feng, Wenlong Lyu, Zhitang Chen, Junjie Ye, Mingxuan Yuan, and Jianye Hao. Batch sequential black-box optimization with embedding alignment cells for logic synthesis. In *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pages 1–9, 2022.
- [3] Jianyong Yuan, Peiyu Wang, Junjie Ye, Mingxuan Yuan, Jianye Hao, and Junchi Yan. Easyso: Exploration-enhanced reinforcement learning for logic synthesis sequence optimization and a comprehensive rl environment. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9, 2023.
- [4] Abdelrahman Hosny, Soheil Hashemi, Mohamed Shalan, and Sherief Reda. Drills: Deep reinforcement learning for logic synthesis. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 581–586, 2020.
- [5] Keren Zhu, Mingjie Liu, Hao Chen, Zheng Zhao, and David Z. Pan. Exploring logic optimizations with reinforcement learning and graph convolutional network. In *2020 ACM/IEEE 2nd Workshop on Machine Learning for CAD (MLCAD)*, pages 145–150, 2020.
- [6] Guanglei Zhou and Jason H. Anderson. Area-driven fpga logic synthesis using reinforcement learning. In *2023 28th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 159–165, 2023.
- [7] Yasasvi V. Peruvemba, Shubham Rai, Kapil Ahuja, and Akash Kumar. RL-guided runtime-constrained heuristic exploration for logic synthesis. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pages 1–9, 2021.
- [8] Yu Qian, Xuegong Zhou, Hao Zhou, and Lingli Wang. An efficient reinforcement learning based framework for exploring logic synthesis. *ACM Trans. Des. Autom. Electron. Syst.*, 29(2), January 2024.
- [9] Luca Gaetano Amarù, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. The epfl combinational benchmark suite. 2015.
- [10] Robert Brayton and Alan Mishchenko. Abc: An academic industrial-strength verification tool. In Tayssir Touili, Byron Cook, and Paul Jackson, editors, *Computer Aided Verification*, pages 24–40, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [11] Mathias Soeken, Heinz Riener, Winston Haaswijk, and Giovanni De Micheli. The EPFL logic synthesis libraries. *CoRR*, abs/1805.05121, 2018.
- [12] Alessandro Tempia Calvino and Giovanni De Micheli. Scalable logic rewriting using don't cares. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6, 2024.
- [13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [14] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [15] Siang-Yun Lee and Giovanni De Micheli. Heuristic logic resynthesis algorithms at the core of peephole optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(11):3958–3971, 2023.
- [16] Heinz Riener, Eleonora Testa, Winston Haaswijk, Alan Mishchenko, Luca Amarù, Giovanni De Micheli, and Mathias Soeken. Scalable generic logic synthesis: One approach to rule them all. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2019.
- [17] Walter Lau Neto, Max Austin, Scott Temple, Luca Amaru, Xifan Tang, and Pierre-Emmanuel Gaillardon. Lsoracle: a logic synthesis framework driven by artificial intelligence: Invited paper. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–6, 2019.
- [18] Alessandro Tempia Calvino, Heinz Riener, Shubham Rai, Akash Kumar, and Giovanni De Micheli. A versatile mapping approach for technology mapping and graph optimization. In *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 410–416, 2022.